

 Please do not reach out to reviewers directly; messaging them will not speed up your review.

# Submission Checklist

Use this checklist before every submission to ensure your task is complete and will pass review.

**For detailed requirements:** See [Task Requirements](#) for complete specifications on each component.

## Pre-Submission Verification

### Task Design

- ☐ Problem statement is clear and unambiguous
- ☐ All requirements are explicitly stated
- ☐ Uses absolute paths (e.g., `/app/file.txt`)
- ☐ Output files are named in instructions
- ☐ Data schemas are fully specified
- ☐ Difficulty target: < 80% pass rate

### Required Files

#### Always required:

- ☐ `task.toml` — Complete configuration with all required sections ([requirements](#))
- ☐ `environment/Dockerfile` — Builds successfully; language dependencies pinned, every `FROM` image digest-pinned, final runtime base canonical (or non-canonical with a justification), and `environment/` within size limits ([requirements](#))

## Non-milestone tasks ( `number_of_milestones = 0` ):

- ☐ `instruction.md` — Clear, human-written instructions ([requirements](#))
- ☐ `environment/Dockerfile` — Builds successfully; application dependencies pinned to exact versions; all base images use `@sha256:<digest>` pins ([requirements](#))
- ☐ `solution/solve.sh` — Deterministic, human-written solution ([requirements](#))
- ☐ `tests/test.sh` — Runs Python pytest tests with pre-installed verifier dependencies and produces reward file ([requirements](#))
- ☐ `tests/test_outputs.py` — Python pytest tests with docstrings that verify behavior

## Milestone tasks ( `number_of_milestones >= 2` ) — see the [Milestones page](#) for the full layout:

- ☐ `task.toml` includes one `[[steps]]` block per milestone (count must equal `number_of_milestones` )
- ☐ No root-level `instruction.md` , `tests/` , `solution/` , or `milestone_x.md` files
- ☐ For each milestone `N` , a `steps/milestone_N/` directory containing:
  - ☐ `instruction.md` — prompt for milestone `N` only (milestone 1 should include the overall task context)
  - ☐ `tests/test.sh` — milestone `N` 's test runner
  - ☐ `tests/test_mN.py` — pytest assertions scored only against milestone `N`
  - ☐ `solution/solve.sh` — wrapper that runs `solveN.sh`
  - ☐ `solution/solveN.sh` — oracle solution scoped only to milestone `N`

## Rubric

- Every submission should include a rubric that is aligned to the task.
- You generate a synthetic rubric via the submission UI in the Snorkel Platform, then edit it for accuracy and completeness.
- The rubric must include **at least three** criteria that assign **negative** rewards (for example, `-1` ).
- See the [Rubrics page](#) for workflow details and quality criteria.

## Quality Standards

- ☐ All requirements have corresponding tests
- ☐ All tests verify described requirements with complete coverage of the prompt (explicit, implicit, and edgecases.)
- ☐ Tests are written in Python and run with pytest
- ☐ Anti-cheating measures in place (no hints or exposed answers)
- ☐ Tests check behavior, not implementation
- ☐ Complies with [Quality Guidelines](#)

## Automated Checks

### Oracle Agent

**BASH** Copy

```
harbor run -a oracle -p <task-folder>
```

- ☐ Oracle agent PASSES

### CI Checks

**BASH** Copy

```
harbor tasks check <task-folder> -m openai/@openai/gpt-5.5
```

- ☐ pinned\_dependencies ✓
- ☐ check\_pinned\_images ✓
- ☐ check\_sanctioned\_base\_images ✓
- ☐ check\_build\_context\_size ✓
- ☐ typos ✓
- ☐ tests\_or\_solution\_in\_image ✓
- ☐ check\_dockerfile\_references ✓
- ☐ check\_test\_sh ✓

- ☐ check\_task\_absolute\_path ✓
- ☐ check\_privileged\_containers ✓
- ☐ ruff ✓
- ☐ check\_task\_sizes ✓
- ☐ validate\_task\_fields ✓

Warnings should also be fixed unless an explicit reviewer-approved exception applies:

- ☐ check\_dockerignore (warn)
- ☐ check\_dockerfile\_hygiene (warn)
- ☐ check\_offline\_tests (warn)
- ☐ check\_apt\_usage (warn)
- ☐ check\_reproducible\_builds (warn)
- ☐ check\_layer\_volatility (warn)
- ☐ check\_no\_build\_tools\_in\_runtime (warn)
- ☐ check\_file\_extraction (warn)
- ☐ check\_heredoc\_usage (warn)
- ☐ check\_recursive\_permissions (warn)

## LLMaJ Checks

- ☐ behavior\_in\_task\_description ✓
- ☐ behavior\_in\_tests ✓
- ☐ informative\_test\_docstrings ✓
- ☐ anti\_cheating\_measures ✓
- ☐ structured\_data\_schema ✓
- ☐ hardcoded\_solution ✓



## Real Agent Testing

## Run Against GPT-5.5

BASHCopy

```
stb harbor run -m @openai/gpt-5.5 -p <task-folder>
```

- ☐ Run 1: PASS / FAIL
- ☐ Run 2: PASS / FAIL
- ☐ Run 3: PASS / FAIL

Run Against Claude Opus 4.8

BASHCopy

```
stb harbor run -m @anthropic/claude-opus-4-8 -p <task-folder>
```

- ☐ Run 1: PASS / FAIL
- ☐ Run 2: PASS / FAIL

Difficulty Calculation

| DIFFICULTY | THRESHOLD                                                                   |
|------------|-----------------------------------------------------------------------------|
| Hard       | Accuracy ≤ 20% on the <b>best</b> model, OR ≤ 20% on the <b>worst</b> model |
| Medium     | 20% < accuracy ≤ 60% on the <b>worst</b> model                              |
| Easy       | 60% < accuracy ≤ 80% on the <b>worst</b> model                              |

- Worst-model pass rate: \_\_\_\_%
- Best-model pass rate: \_\_\_\_%
- Difficulty: Easy / Medium / Hard

Tasks where the worst model scores above 80% will not be accepted.  
Full criteria: [Difficulty Guidelines](#).

## Final Review

Use the [Reviewer Checklist](#) to validate your task against the same criteria reviewers use before you submit.

## Self-Check Questions

### 1. Would I understand this task as a first-time reader?

- If no → Clarify instructions

### 2. Are there any ambiguous requirements?

- If yes → Make them explicit

### 3. Could an agent cheat on this task?

- If yes → Add anti-cheating measures

### 4. Do tests verify actual behavior?

- If no → Rewrite to test behavior

### 5. Is the solution deterministic?

- If no → Add seeds, remove randomness

## Submission Method

- ☐ Created ZIP of files (not folder)
- ☐ All required files included in ZIP
- ☐ Uploaded to terminus-project-v2 on Snorkel Expert Platform
- ☐ Metadata filled in

See [Platform Submission Guide](#) for detailed submission steps.

## Ready?

If you've completed all items above, upload your ZIP file to the Snorkel Expert Platform.

**Good luck!** 🎉

## Need Help?

- Slack: [#ec-terminus-submission](#)
- [Reviewer Checklist](#)
- [Troubleshooting guide](#)
- [FAQ](#)

---

PREVIOUS

[Platform Submission](#)

NEXT

[After Submission](#)